

Tutorial - Semana 2 - Encontro 2

Introdução

No Encontro 1 você construiu o `BP_Player` como subclasse de `Character` e ajustou seus parâmetros de locomoção, mas ele ainda depende do esquema de controle padrão do template — as teclas de movimento e o mouse já vêm mapeados de fábrica, sem que você tenha decidido nada sobre esse mapeamento. Este encontro resolve essa dependência: você vai substituir o controle padrão por um esquema próprio de **Enhanced Input**, o sistema que desacopla a intenção do jogador ("mover", "olhar") do dispositivo físico que a aciona (tecla, botão, eixo de analógico).

Este tutorial não substitui a explicação do professor em sala. Ele existe para que você possa acompanhar a implementação passo a passo durante o laboratório e revisar os passos depois da aula, sem depender da documentação oficial da Epic.

Objetivos da Semana

- Explicar Enhanced Input (Input Actions, Input Mapping Contexts, Triggers, Modifiers) como camada de desacoplamento entre a intenção do jogador e a ação no mundo.
- Configurar Enhanced Input para movimentação e câmera do `BP_Player`.
- Implementar uma nova Input Action não demonstrada em aula, com liberdade de solução.

Resultado Esperado ao Final da Semana

O `BP_Player` controlável por um esquema próprio de Enhanced Input (movimentação e câmera), acrescido de ao menos uma Input Action adicional resultante do desafio (correr, agachar ou pular), funcional no `Map_Exploration`. Combinado ao resultado do Encontro 1, a Semana 2 termina com o `BP_Player` totalmente independente do controle padrão do template.

Pré-requisitos

- Ter concluído o Encontro 1: `BP_Player` criado como `Character`, com parâmetros de movimento ajustados e posicionado no `Map_Exploration`.
 - Compreender a diferença entre `Pawn` e `Character` e o papel do `Character Movement Component`.
-

Antes de começar

O que você deverá possuir antes desta semana

- O projeto do Encontro 1, aberto no Unreal Editor 5.6, com o `BP_Player` funcional e controlável pelo esquema padrão do template.

Arquivos necessários

- Nenhum arquivo externo é necessário neste encontro.

Assets utilizados

- Nenhum asset externo é necessário. Os Input Actions e Input Mapping Context são criados inteiramente dentro do editor nesta aula.

Projeto esperado

O mesmo projeto do Encontro 1, com o `BP_Player` pronto para receber sua própria lógica de input, substituindo a dependência do esquema padrão gerado pelo template Third Person.

Parte 1 — Input Actions e Input Mapping Context

Objetivo

Compreender o desacoplamento entre dispositivo físico e intenção de jogo, e criar as Input Actions de movimentação e câmera.

Conceito

Todo sistema de input moderno precisa resolver o mesmo problema: a lógica de gameplay nunca deveria depender diretamente de qual tecla, botão ou eixo de controle foi pressionado. Se o `BP_Player` verificasse diretamente "a tecla W foi pressionada?", qualquer mudança de dispositivo (teclado para controle, remapeamento de teclas, acessibilidade) exigiria reescrever a lógica de movimentação inteira.

O Enhanced Input resolve isso em camadas:

- Uma **Input Action** representa uma intenção abstrata do jogador — "mover", "olhar", "pular" — completamente independente de qual tecla ou botão a aciona.
- Um **Input Mapping Context** define o mapeamento concreto entre um dispositivo físico (tecla, botão, eixo) e uma Input Action. Um projeto pode ter múltiplos Mapping Contexts, ativados ou combinados em tempo real (por exemplo, um esquema para exploração a pé e outro para dirigir um veículo).
- Um **Trigger** determina quando um valor de input é considerado acionado: pressionado uma vez, mantido, solto, duplo toque.
- Um **Modifier** altera o valor bruto do input antes de chegar à lógica de gameplay: inversão de eixo, zona morta (dead zone), escala de sensibilidade.

Esse desacoplamento é o que garante que a lógica do `BP_Player` sempre responda a "mover para frente", nunca a "tecla W", independentemente do dispositivo por trás.

Passo a passo

1. No Content Browser, criar a subpasta `Input` dentro de `Blueprints/Characters` (ou em uma pasta de Input própria, conforme preferir organizar).
2. Clicar com o botão direito, selecionar "Input" e criar uma "Input Action". Nomear como `IA_Move`.
3. Repetir o processo para criar `IA_Look` (câmera/direção do olhar) e `IA_Jump` (salto).
4. Abrir `IA_Move` e, no painel Details, configurar o "Value Type" como "Axis2D (Vector2D)", já que movimentação envolve duas direções simultâneas (frente/trás, esquerda/direita).
5. Abrir `IA_Look` e configurar o "Value Type" também como "Axis2D (Vector2D)" (horizontal e vertical da câmera).
6. Abrir `IA_Jump` e manter o "Value Type" como "Digital (bool)", já que pular é uma ação de pressionar/soltar.
7. Criar um "Input Mapping Context" na mesma pasta, nomeado como `IMC_Player`.
8. Abrir `IMC_Player` e adicionar um mapeamento para `IA_Move`, associando as teclas W/A/S/D (ou o eixo correspondente) aos eixos X/Y da ação.
9. Adicionar um mapeamento para `IA_Look`, associando o movimento do mouse aos eixos X/Y da ação.
10. Adicionar um mapeamento para `IA_Jump`, associando a barra de espaço.

Resultado esperado

Três Input Actions (`IA_Move` , `IA_Look` , `IA_Jump`) e um Input Mapping Context (`IMC_Player`) criados no Content Browser, com os mapeamentos de teclado e mouse configurados, mas ainda não conectados à lógica do `BP_Player` .

Verificando

Abra `IMC_Player` e confirme visualmente que as três Input Actions aparecem listadas, cada uma com seu mapeamento de tecla ou eixo correspondente, sem erros ou avisos no editor.

Problemas comuns

- **Value Type incorreto:** configurar `IA_Move` ou `IA_Look` como "Digital (bool)" em vez de "Axis2D (Vector2D)" impede movimentação em duas dimensões; revise o Value Type de cada Input Action antes de prosseguir.
- **Esquecer de nomear com o prefixo `IA_` ou `IMC_` :** viola a convenção de nomenclatura do projeto e dificulta localizar os assets mais adiante — renomeie antes de continuar.
- **Mapear a mesma tecla em duas Input Actions diferentes:** gera conflito de input; cada tecla deve corresponder a uma única intenção dentro do mesmo Mapping Context.

Boas práticas

Nomeie Input Actions pela intenção que representam (`IA_Move` , `IA_Jump`), nunca pelo dispositivo ou tecla (evite nomes como `IA_Tec law`) — esse é o princípio central do desacoplamento que esta aula ensina.

Comparação com Unity

Enhanced Input corresponde ao Input System (novo) da Unity: ambos abstraem o dispositivo físico em Actions e permitem múltiplos esquemas de controle combináveis. A diferença arquitetural mais relevante é que a Unreal organiza esses esquemas em Input Mapping Contexts que podem ser adicionados, removidos ou priorizados dinamicamente em runtime (por exemplo, trocar de esquema ao entrar em um veículo), enquanto a Unity resolve um problema equivalente por meio de Action Maps dentro de um único Input Actions Asset, ativados e desativados via código.

Parte 2 — Conectando o Enhanced Input ao BP_Player

Objetivo

Conectar o `IMC_Player` e as três Input Actions à lógica de movimentação, câmera e salto do `BP_Player`, substituindo o controle padrão do template.

Conceito

Criar as Input Actions e o Mapping Context não é suficiente: é preciso, primeiro, dizer à engine que o `BP_Player` deve usar aquele Mapping Context específico ao ser controlado, e depois traduzir cada evento de Input Action disparado em uma chamada real de movimentação, rotação de câmera ou salto sobre o Character Movement Component herdado no Encontro 1.

Imagem sugerida

Objetivo: mostrar o Event Graph do `BP_Player` com os nós de Enhanced Input conectados às funções de movimento. Enquadramento: captura do Event Graph do Blueprint, centralizada nos nós Enhanced Input Action Move/Look/Jump. Elementos importantes: nó "Add Mapping Context" no Event BeginPlay; nós "Enhanced Input Action" para `IA_Move`, `IA_Look` e `IA_Jump`; conexões para Add Movement Input, Add Controller Yaw/Pitch Input e Jump. O que deve ser destacado: o fluxo desde o evento de input abstrato (`IA_Move`) até a chamada concreta de movimentação (Add Movement Input). Legenda sugerida: "Do dispositivo físico à ação no mundo: o fluxo completo do Enhanced Input no `BP_Player`." Referência visual (documentação oficial, apenas para consulta — não copiar a imagem): [Enhanced Input in Unreal Engine](#)

Passo a passo

1. Abrir o `BP_Player` e ir até o Event Graph.
2. No Event `BeginPlay` (ou criando um se ainda não existir), adicionar um nó "Get Player Controller", seguido de "Get Enhanced Input Local Player Subsystem" e "Add Mapping Context", referenciando `IMC_Player`.
3. No painel Class Defaults ou diretamente no Event Graph, adicionar um nó "Enhanced Input Action" para `IA_Move` (clicando com o botão direito no grafo e buscando pelo nome da ação).
4. Conectar a saída "Triggered" do nó `IA_Move` a dois nós "Add Movement Input": um usando o vetor "Forward" do Controller multiplicado pelo eixo Y do valor da ação, outro usando o vetor

"Right" multiplicado pelo eixo X.

5. Adicionar um nó "Enhanced Input Action" para `IA_Look` e conectar sua saída "Triggered" aos nós "Add Controller Yaw Input" (eixo X) e "Add Controller Pitch Input" (eixo Y).
6. Adicionar um nó "Enhanced Input Action" para `IA_Jump` e conectar sua saída "Triggered" à função "Jump" (herdada do Character), e a saída "Completed" à função "Stop Jumping".
7. Compilar o Blueprint e salvar.
8. Testar em modo Play no `Map_Exploration`: mover com WASD, olhar ao redor com o mouse e pular com a barra de espaço.

Resultado esperado

O `BP_Player` se move, gira a câmera e pula exclusivamente através do esquema `IMC_Player` configurado por você — o controle padrão gerado pelo template não está mais em uso, mesmo que os mapeamentos de tecla sejam parecidos.

Verificando

Remova temporariamente o nó "Add Mapping Context" do BeginPlay (ou desative-o) e teste em Play: se o personagem parar de responder a qualquer input, isso confirma que o controle depende inteiramente do `IMC_Player` configurado, não de um resquício do template. Reative o nó em seguida.

Problemas comuns

- **Personagem não se move mesmo com o Mapping Context adicionado:** confirme se o "Add Mapping Context" está de fato sendo executado no BeginPlay (adicione um Print String temporário para verificar) e se os nós "Enhanced Input Action" no grafo referenciam exatamente `IA_Move`, `IA_Look` e `IA_Jump` criados na Parte 1.
- **Câmera gira ao contrário do esperado:** ajuste um Modifier de "Negate" em `IA_Look` no próprio `IMC_Player`, em vez de inverter a lógica no Event Graph.
- **Movimentação funciona apenas em uma direção:** confirme que os nós "Add Movement Input" usam os vetores corretos (Forward para o eixo Y, Right para o eixo X) e que ambos os eixos de `IA_Move` estão mapeados no `IMC_Player`.

Boas práticas

Organize os nós de Enhanced Input no Event Graph dentro de uma Comment Box nomeada "Input", separando visualmente essa lógica da lógica de outros sistemas que serão adicionados ao

`BP_Player` nos módulos seguintes (Interaction, Inventory, Health) — hábito que evita o Event Graph gigante já mencionado como boa prática no `PROJECT_ARCHITECTURE.md`, seção 10.

Comparação com Unity

Enhanced Input corresponde ao Input System (novo) da Unity: ambos abstraem o dispositivo físico em Actions e permitem múltiplos esquemas de controle combináveis. A diferença arquitetural mais relevante é que a Unreal organiza esses esquemas em Input Mapping Contexts que podem ser adicionados, removidos ou priorizados dinamicamente em runtime (por exemplo, trocar de esquema ao entrar em um veículo), enquanto a Unity resolve um problema equivalente por meio de Action Maps dentro de um único Input Actions Asset, ativados e desativados via código.

Parte 3 — Desafio: uma nova Input Action

Objetivo

Aplicar de forma autônoma o fluxo completo de Enhanced Input (Input Action → Input Mapping Context → lógica no Event Graph), implementando uma ação não demonstrada em aula.

Conceito

Esta etapa não introduz um conceito novo — ela testa se o fluxo de desacoplamento entre dispositivo e intenção foi de fato compreendido, pedindo que você resolva um pequeno problema de decisão técnica sem um passo a passo fechado. É o primeiro exercício de decisão autônoma da disciplina em escopo estreito e seguro (`PEDAGOGICAL_RULES.txt`), mas exige que você escolha sozinho o Trigger e o Modifier apropriados para a ação escolhida.

Passo a passo

1. Escolher uma entre três ações: correr (sprint), agachar (crouch) ou uma variação de salto (por exemplo, pulo mais alto ao segurar o botão).
2. Criar uma nova Input Action na pasta `Input` (por exemplo, `IA_Sprint`), com o Value Type apropriado (Digital para uma ação de pressionar/soltar).
3. Adicionar o mapeamento da nova ação em `IMC_Player`, escolhendo uma tecla livre (por exemplo, Left Shift para sprint, Left Ctrl para agachar).

4. Configurar, se necessário, um Trigger apropriado (por exemplo, "Hold" para uma ação que precisa ser mantida pressionada) e um Modifier, se fizer sentido para o comportamento escolhido.
5. No Event Graph do `BP_Player`, adicionar o nó "Enhanced Input Action" correspondente e conectar sua lógica à propriedade relevante do Character Movement Component (por exemplo, alterar "Max Walk Speed" dinamicamente para o sprint, ou alterar o "Half Height" do Capsule Component para o agachar).
6. Compilar, salvar e testar em modo Play.

Resultado esperado

Uma nova ação de gameplay funcional, acionada por uma tecla própria mapeada no `IMC_Player`, produzindo um efeito perceptível na movimentação do `BP_Player` (velocidade aumentada, capsula reduzida, ou salto diferenciado).

Verificando

Compare sua solução com a de um colega: as escolhas de Trigger, Modifier e forma de implementação devem poder ser diferentes entre si, já que o desafio não tem gabarito único.

Problemas comuns

- **Referenciar a tecla diretamente na lógica em vez de criar uma Input Action nomeada pela intenção:** contraria o princípio de desacoplamento ensinado nesta aula; sempre crie uma Input Action própria (`IA_Sprint`, não uma checagem direta de tecla).
- **Esquecer de adicionar o novo mapeamento ao `IMC_Player` já em uso:** a nova ação não funciona se estiver apenas na Input Action, sem o mapeamento correspondente no Mapping Context ativo.

Boas práticas

Documente, em uma Comment Box no Event Graph, qual Trigger e Modifier você escolheu para a nova ação e por quê — esse hábito de justificar decisões técnicas será formalmente avaliado a partir do Módulo 2, nos Desafios Técnicos (Rubrica 2).

Comparação com Unity

A mesma decisão — adicionar uma nova ação de sprint ou agachar —, na Unity, seria resolvida criando uma nova Action dentro do Input Actions Asset e mapeando-a em um Action Map já em uso, com a lógica correspondente lida via código C# (por exemplo, `OnSprint(InputAction.CallbackContext context)`). O princípio de nomear pela intenção, não pela tecla, é o mesmo nas duas engines.

Ao final da semana

Ao final da Semana 2 (Encontros 1 e 2), o `BP_Player` deve estar completo como: subclasse de `Character`, com parâmetros de movimento ajustados (Encontro 1), controlável por um esquema próprio de Enhanced Input para movimentação e câmera, e acrescido de uma Input Action adicional resultante do desafio (Encontro 2). Isso corresponde à conclusão das linhas "BP_Player (locomoção)" e "Input do jogador" do roadmap do Módulo 1 no `PROJECT_ARCHITECTURE.md` — a base sobre a qual o graybox de terreno e materiais da Semana 3 será construído, e sobre a qual o `GameMode` e o `PlayerController` da Semana 4 vão se apoiar.

Desafio

O desafio desta semana é a própria Parte 3: adicionar uma nova Input Action não demonstrada em aula (correr, agachar ou pular de forma diferenciada), com liberdade de implementação quanto ao Trigger, ao Modifier utilizado e à forma como a ação se manifesta no `BP_Player`. Diferentes soluções são esperadas e aceitas.

Checklist

- ☐ `IA_Move`, `IA_Look` e `IA_Jump` criados com o Value Type correto
- ☐ `IMC_Player` criado, com os três mapeamentos configurados
- ☐ `Add Mapping Context` presente no `BeginPlay` do `BP_Player`, referenciando `IMC_Player`
- ☐ Movimentação, câmera e salto funcionais exclusivamente via Enhanced Input, sem depender do controle padrão do template
- ☐ Nova Input Action do desafio (`IA_Sprint` ou equivalente) implementada e funcional
- ☐ Blueprint compilado sem erros e salvo

Glossário

- **Enhanced Input:** sistema da Unreal que desacopla o dispositivo físico da intenção de gameplay por meio de Input Actions e Input Mapping Contexts.
- **Input Action:** representação abstrata de uma intenção do jogador (mover, olhar, pular), independente do dispositivo que a aciona.
- **Input Mapping Context:** conjunto de mapeamentos entre dispositivos físicos e Input Actions, que pode ser adicionado, removido ou priorizado em tempo real.
- **Trigger:** regra que determina quando um valor de input é considerado acionado (pressionado, mantido, duplo toque).
- **Modifier:** transformação aplicada ao valor bruto do input antes de chegar à lógica de gameplay (inversão, zona morta, escala).

Referências

- EPIC GAMES. **Unreal Engine 5 Documentation** — Enhanced Input in Unreal Engine. Disponível em: <https://dev.epicgames.com/documentation/en-us/unreal-engine/enhanced-input-in-unreal-engine>.
- EPIC GAMES. **Unreal Engine 5 Documentation** — Gameplay Framework in Unreal Engine. Disponível em: <https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-framework-in-unreal-engine>.
- EPIC GAMES. **Unreal Engine Learning Library** — introdução a Enhanced Input. Disponível em: <https://dev.epicgames.com/community/unreal-engine/learning>.
- UNITY TECHNOLOGIES. **Unity Manual** — Input System, para fins comparativos. Disponível em: <https://docs.unity3d.com/Manual/>.
- Vídeo sugerido (apoio complementar, nunca substituindo a documentação oficial): canal oficial **Unreal Engine**, vídeos introdutórios de Enhanced Input.